

>_ CLAUDE CODE GLOSSARY • 2026

Introduction to Claude Code in 30 terms

From context windows and CLAUDE.md to hooks, subagents, RAG and guardrails — the working vocabulary for anyone building with Claude Code. 📌



Ashwini

Senior AI/ML Engineer

Four ideas that set the frame

Before hooks or subagents make sense, these four decide how Claude Code sees your project and this conversation:



Context Window

Shared memory for the chat — prompts, code and replies all take up space in it.



CLAUDE.md

A project handbook Claude reads first: conventions, structure, house rules.



MCP

One shared protocol to plug into GitHub, Slack, databases and APIs.



Subagents

Helper agents that split a big task into pieces, then report back.



Get these four right and every other term in this deck builds on top of them.



Ashwini

Senior AI/ML Engineer

Plan first, automate the rest



Hooks

Small programs that fire on events — tests, formatting, security checks.



Plan Mode

Drafts steps, files and risks before touching anything — you approve first.



Skills

A reusable workflow Claude runs the same way, every time it's needed.



Checkpoints

A saved snapshot you can return to the moment something breaks.



Plan Mode stops a bad idea before it runs; Checkpoints undo it if one slips through anyway.



Ashwini

Senior AI/ML Engineer

What you type is what it costs

Every exchange is built from the same four pieces, and each one is billed and remembered separately:



Tokens

The word-pieces AI reads text in — they set cost, speed and memory use.



System Prompt

Sets the AI's role and behavior — applied to every single response.



Prompt

The question or instruction you send — clearer prompts, better answers.



Prompt Engineering

Structuring prompts on purpose — context, format, examples — for better output.



The system prompt sets the rules once; your prompt, and how well you engineer it, decides the result every time.



Ashwini

Senior AI/ML Engineer

Same model, different answers

None of these change what the model knows — they change how it picks words and how you receive them:



Temperature

The creativity dial — low stays factual, high gets more inventive.



Top-P

Limits next-word choices to the most likely pool — lower plays it safer.



Streaming

The answer appears gradually, word by word, instead of all at once.

Temperature and Top-P decide what gets said; streaming decides how soon you see it.



Ashwini

Senior AI/ML Engineer

From answering to actually doing

A plain model can only describe what it would do. Tool use is what lets it actually do it:

Text-only model

- ✗ Answers only from training data
- ✗ Can't fetch anything live
- ✗ Can't touch your systems

Tool Use + Function Calling

- ✓ Tool Use — reaches search, databases, code execution
- ✓ Function Calling — triggers one predefined function directly
- ✓ Structured actions instead of free text

Function calling is how tool use actually gets wired in — a button the model can press, not just describe.



Ashwini

Senior AI/ML Engineer

How AI finds your own data

Three ideas stack on top of each other to let an LLM answer from documents it was never trained on:



- 1 **Embeddings** — text turned into numbers that capture meaning
- 2 **Vector Database** — stores those numbers, searchable by similarity
- 3 **RAG** — retrieves the relevant chunks, then generates the answer

Embeddings make meaning searchable; RAG is what turns that search into a grounded answer.



Ashwini

Senior AI/ML Engineer

Better answers come from more steps

A knowledge base gives the model something to check; these two techniques decide how carefully it uses it:



Knowledge Base

A searchable set of your own docs and data the AI can answer from.



Reflection Loop

Draft, review, improve – the AI edits its own answer before you see it.



Chain of Thought

Reasoning broken into visible steps before landing on the final answer.

A knowledge base gives the model facts to check; reflection and chain of thought make it actually check them.



Ashwini

Senior AI/ML Engineer

Deciding what happens, and how

Once an agent has multiple tools and multiple steps, something has to keep the run on track:



Planning Agent

Breaks the task into a to-do list first, then works through it in order.



Tool Routing

Decides which of the agent's tools is right for this particular step.



Guardrails

Rules that block harmful or off-policy output before it reaches you.

SAFETY IS NOT OPTIONAL

A planning agent with no guardrails is still just a fast way to do the wrong thing at scale.

Planning decides the order, routing decides the tool, guardrails decide what's off the table entirely.



Ashwini

Senior AI/ML Engineer

The agent, and what keeps it in check

An agent doesn't run in a vacuum — these three limits shape how it's allowed to operate:



Agent

Plans, uses tools, executes steps and works toward a goal on its own.



Sandbox

An isolated environment where code runs without touching your real system.



Rate Limit

A cap on requests per window, so no one user overloads the service.



Latency

The delay between sending a request and getting a response back.



An agent is only as trustworthy as the sandbox, rate limit and latency budget it's allowed to run inside.



Ashwini

Senior AI/ML Engineer

🕒 BEFORE YOUR NEXT SESSION

Which of these 30 were new to you?

Context windows, hooks, subagents, RAG, guardrails – this is the working vocabulary for anyone building with Claude Code today. Save it before your next session, share it with your team, and follow for more breakdowns like this.

Repost

Save

Comment

 Follow Ashwini

#ClaudeCode #AIEngineering #PromptEngineering #RAG #AIAgents



Ashwini

Senior AI/ML Engineer